



Control Theory - State Estimation

Manan Garg

MPC - AeRoVe

What is state estimation?



Till now, we had –

$$\begin{aligned}\dot{x} &= Ax + Bu \\ u &= -Kx\end{aligned}$$

We assumed that – we **know** (i.e. can **measure**) the state x at all times

(Not possible in real cases!!!)

What is state estimation?

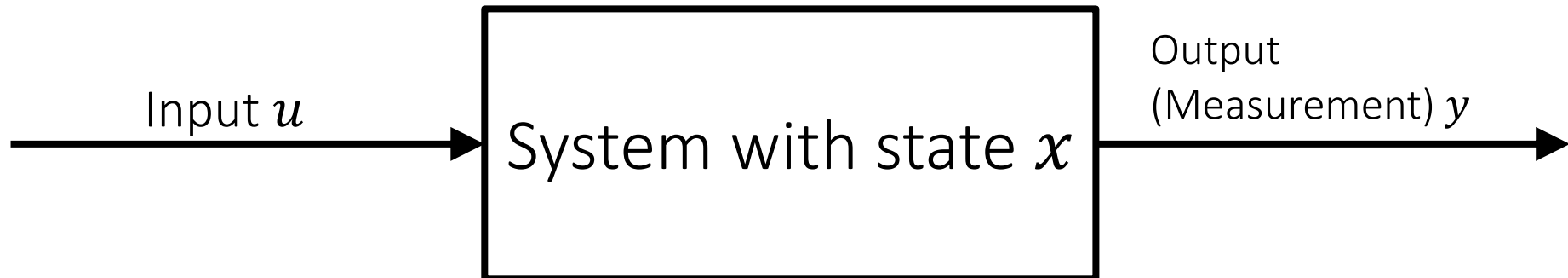
Realistically, we can only **measure** y , which might **not** be able to convey complete information about the **full state** x .

So our system becomes,

$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx\end{aligned}$$

where, y = measurement output vector

x = full state vector



What is state estimation?

$$y = Cx$$

State estimation – Come up with an estimate $\hat{x}$ for the **full** state, using –

- 1) Our model (A, B) of the system
- 2) Measurements y made on the system

Consider a drone moving only up or down (in z-direction).



Say, we are interested in controlling the state –

$$x = \begin{bmatrix} \textit{position} \\ \textit{velocity} \end{bmatrix}$$

Say, that the drone is in an environment that offers a viscous damping force ($\textit{acceleration} = -b * \textit{velocity}$). Then the system can be written as –

$$\dot{x} = \begin{bmatrix} \textit{velocity} \\ \textit{acceleration} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -b \end{bmatrix} \begin{bmatrix} \textit{position} \\ \textit{velocity} \end{bmatrix} = Ax$$

However, the drone only has an IMU sensor that can only measure acceleration of the drone.

$$y = [\textit{acceleration}] = \begin{bmatrix} 0 & -b \end{bmatrix} \begin{bmatrix} \textit{position} \\ \textit{velocity} \end{bmatrix} = Cx$$

Need to estimate x , given model and measurement $y \Rightarrow$ **State estimation!**

Kalman Filter



Goal of Full-State-Estimation –

Drive the estimate $\hat{x}$ to the true state x .

In other words, drive the error $\epsilon = x - \hat{x}$ to zero as time $t \rightarrow \infty$.

The equation for $\hat{x}$ that achieves this is -

$$\frac{d}{dt}\hat{x} = A\hat{x} + Bu + K_f(y - C\hat{x})$$

Kalman Filter



$$\frac{d}{dt}\hat{x} = A\hat{x} + Bu + K_f(y - C\hat{x})$$

This equation simplifies to give -

$$\frac{d}{dt}\epsilon = (A - K_f C)\epsilon$$

Note that this equation is completely analogous to the state-controller equation –

$$\dot{x} = (A - K_r B)x$$

Only here, we are driving ϵ to zero, whereas in the controller, we were driving x to zero.

Observability



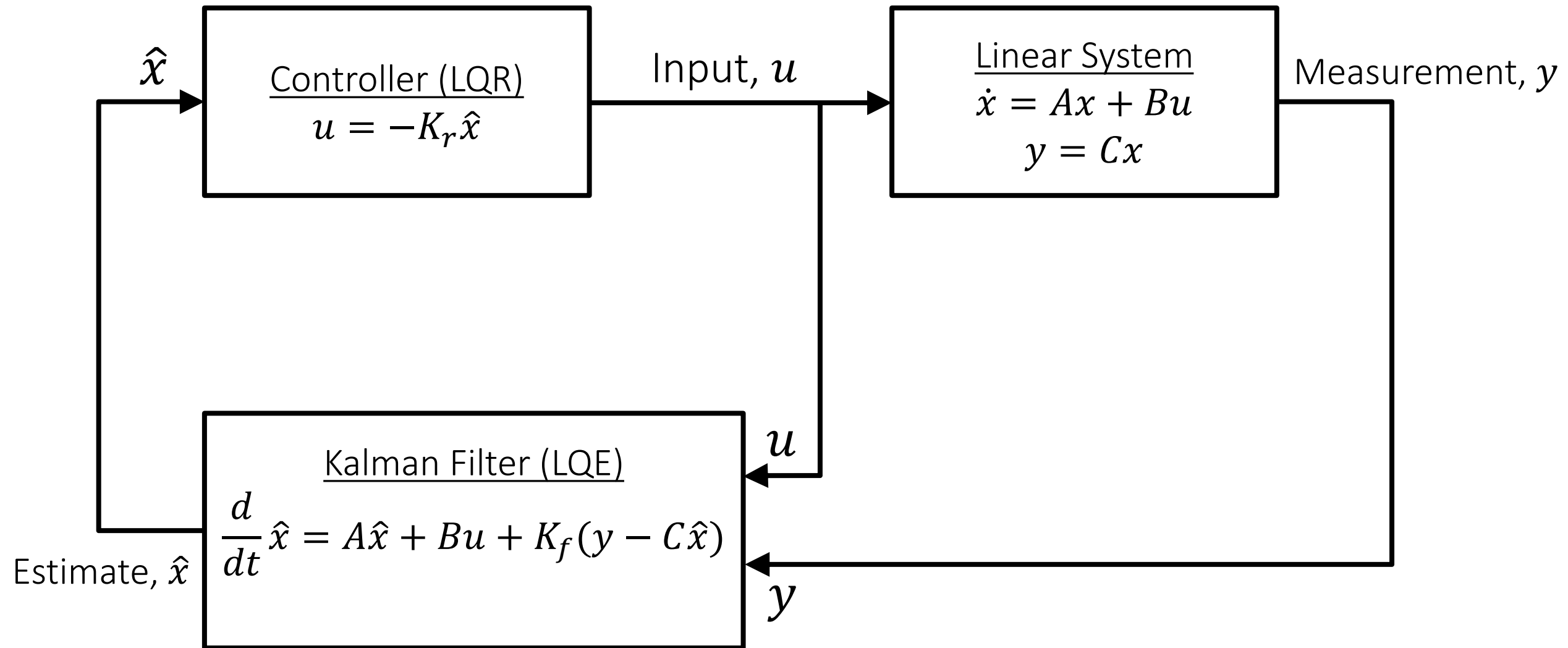
$$\frac{d}{dt}\epsilon = (A - K_f C)\epsilon$$

A system (A, C) is called **observable**, if with an appropriate choice of K_f ,

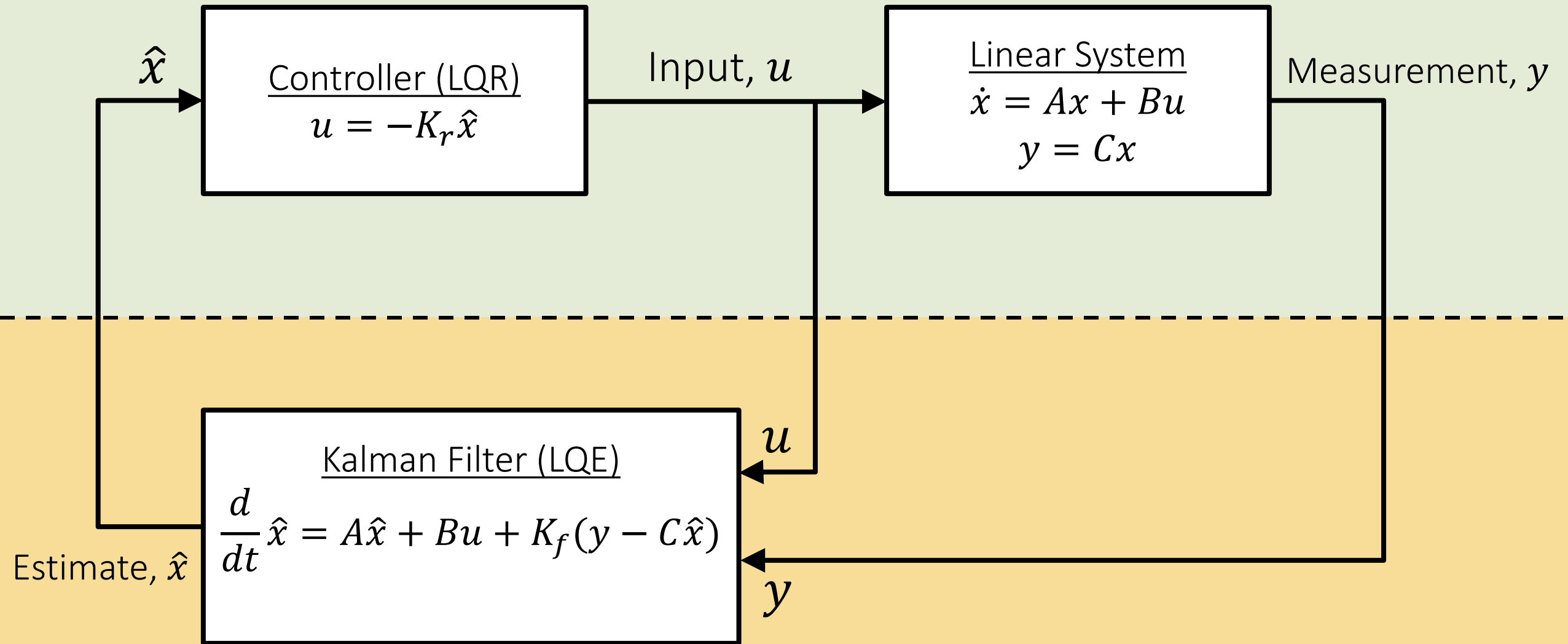
- 1) we can drive ϵ to zero, i.e., our estimate gets closer to the true value with time.
- 2) We can place the eigenvalues of $(A - K_f C)$ anywhere in complex plane.

To test for observability, we define the observability matrix $O = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$

The system is observable if and only if $\text{rank}(O) = n$.



State Controller



State Estimator

Interpreting the Kalman Filter equation

$$\frac{d}{dt}\hat{x} = A\hat{x} + Bu + K_f(y - C\hat{x})$$

The term $A\hat{x} + Bu$ is the **model-prediction** (also called the a priori estimate).

The term $K_f(y - C\hat{x})$ is the **measurement-update**. It updates the a priori estimate proportional to the difference between the actual new measurement y , and the predicted measurement $C\hat{x}$.

Handling noise/disturbances

2 types of noise –

- 1) Process noise/disturbances (w_d) – **Disturbances** in the system, or **inaccuracies of our model** from the true system. E.g., a slight gust of wind pushes the drone slightly in some direction etc.
- 2) Measurement noise (w_n) – Noise in the **measurements** made by the sensors. E.g., Slight error in measurement made by IMU etc.

$$\begin{aligned}\dot{x} &= Ax + Bu + w_d \\ y &= Cx + w_n\end{aligned}$$

We assume that both the noises are Gaussian, and we can find the variances of both (v_d and v_n respectively).

Given the variances v_d and v_n , we might be more confident in our **model** instead of our **measurements**, or vice-versa. We can choose an value of K_f appropriately based on our relative confidence in the model and measurement.

Linear Quadratic Estimator (LQE)

The Linear Quadratic Estimator is an algorithm to determine the optimal K_f , for given variances v_d and v_n .

The LQE implemented along with the LQR is called a LQG (Linear Quadratic Gaussian).

MATLAB Implementation

```
>> obsv(A, C)
```

Returns the observability matrix of the system (A, C)

```
>> rank(obsv(A, C))
```

Returns the rank of the observability matrix

```
>> K = lqe(A, C, v_d, v_n)
```

Gives the optimal K_f that will give optimal convergence of estimate $\hat{x}$ to x .